SUPPLYSYNC

ASRA ZAFAR

Contents

- 1. Abstract
- 2. Introduction
- 3. Problem Statement
- 4. Objectives and Scope
- 5. System Requirements
- 6. Database Design
- 7. System Modules and Features
- 8. SQL Concepts Implemented
- 9. Business Queries and Views
- 10. Data Integrity and Verification
- 11. Execution Guide
- 12. Limitations and Future Enhancements
- 13. Conclusion
- 14. References

Note: This report is based on the supplied SQL implementation file, wholesale_distribution_system.sql. It describes the implemented database objects and query logic; it does not claim measured performance results or a live production deployment.

1. Abstract

SupplySync is a MySQL-based Wholesale Distribution Management System designed to support a business that purchases products from suppliers, stores them in warehouses and distributes them to institutional and business customers. The database brings supplier information, product catalogues, warehouse stock, purchase orders, shipment details, expiry batches and stock movements into one relational system.

The implementation focuses on operational visibility and decision support. SQL views and queries identify stock below reorder levels, rank supplier-product options, estimate potential stock-out dates from recent outgoing movements, monitor pending purchase orders, calculate supplier delivery performance, compare warehouse stock and analyse transportation costs. The script also demonstrates relational constraints, joins, subqueries, window functions, indexes, EXPLAIN, CRUD operations and transaction control.

The project is implemented as a database prototype using MySQL 8.0+ and the InnoDB storage engine. It uses a reproducible reporting date of 10 November 2026 through the helper function fn_today().

2. Introduction

Wholesale distribution involves coordinating multiple suppliers, products, storage locations, customer orders and transport arrangements. When these activities are recorded separately, it becomes difficult to maintain accurate stock balances, identify replenishment needs, monitor delayed deliveries and compare supplier performance.

SupplySync addresses these information-management needs through a centralized relational database. The system models the main entities involved in wholesale distribution and links them through primary and foreign keys. The Stock_Movements table acts as a ledger of goods entering and leaving warehouses, while Inventory stores the current quantity for each warehouse-product combination.

The supplied SQL script creates the database wholesale_db, defines the schema, inserts sample records, creates analytical views, runs verification queries and demonstrates database operations.

3. Problem Statement

A wholesale distributor needs reliable information about what stock is available, where it is stored, which supplier can replenish it, whether incoming orders are delayed and how much delivery operations cost. Without a structured database, duplicate records, inconsistent quantities, missed reorder points and limited supplier visibility can affect day-to-day decisions.

The project therefore develops a relational database that organizes distribution data and provides SQL-based queries for monitoring inventory, suppliers, orders, warehouses and logistics.

4. Objectives and Scope

4.1 Objectives

- Maintain structured records of suppliers, manufacturers, products, warehouses, customers and transport methods.
- Track stock quantities by product and warehouse, including reorder levels.
- Record batches and expiry dates for perishable products.
- Manage purchase orders, order line items, received quantities and shipment status.
- Maintain an IN/OUT stock-movement ledger for stock reconciliation.
- Identify low-stock items and estimate stock-out timing using recent outgoing movements.
- Compare supplier options using quoted price, lead time and quality score.

- Analyse pending orders, delivery performance, warehouse stock and transport costs.
- Demonstrate core DBMS concepts through SQL statements and database objects.

4.2 Scope

The current implementation is a database-level prototype. It provides schema, sample data, views and SQL query logic. It does not include a graphical user interface, user authentication layer, live supplier APIs, automatic notifications, route-mapping integration or machine-learning forecasting.

5. System Requirements

Category	Requirement
Database	MySQL 8.0 or later; InnoDB storage engine; utf8mb4 character set.
Execution environment	MySQL Workbench, MySQL command-line client or a compatible SQL editor/extension.
Input data	Supplier and manufacturer details, product data, stock levels, reorder levels, orders, shipment records and stock movements.
Output	SQL result sets, saved views, stock alerts, supplier rankings, order tracking and analytical summaries.

6. Database Design

The SQL file creates 13 relational tables in the wholesale_db database. Primary keys uniquely identify records, while foreign keys connect related entities and help prevent orphan records.

Table	Purpose
Suppliers	Stores supplier name, location, rating and active status.
Manufacturers	Stores product-manufacturer details.
Products	Stores product name, category, selling price, unit, perishability and manufacturer.
Warehouses	Stores warehouse name, location and storage capacity.
Customers	Stores customer name, type and location.
Transport_Methods	Stores transport type and typical speed.
Supplier_Products	Associative table connecting suppliers and products, with unit price, lead time and quality score.
Inventory	Stores current quantity and reorder level for each warehouse-product pair.
Inventory_Batches	Stores batch number, quantity and expiry date for perishable stock.
Purchase_Orders	Stores supplier, destination warehouse, order dates and order status.
Order_Items	Stores products and quantities included in each purchase order, including received quantity.
Logistics	Stores shipment method, distance, transport cost, dispatch date, delivery date and status.
Stock_Movements	Records stock IN and OUT events, dates, related order/customer and remarks.

6.1 Main relationships

- One manufacturer can be associated with multiple products; each product references a manufacturer.
- Suppliers and products have a many-to-many relationship resolved by Supplier_Products.
- Inventory connects products to warehouses and enforces one row per warehouse-product pair.

- Inventory_Batches references the matching warehouse-product inventory pair.
- Purchase_Orders references a supplier and destination warehouse; Order_Items stores its product lines.
- Logistics stores one shipment per purchase order through a unique order_id.
- Stock_Movements records warehouse-product movements and can reference the related purchase order or customer.

7. System Modules and Features

Inventory monitoring

The Inventory table stores current stock and reorder levels. The low-stock view identifies quantities below the configured threshold and calculates the shortfall.

Supplier evaluation

Supplier_Products stores supplier-specific price, lead time and quality score. The supplier-ranking view/query compares available supplier-product options.

Stock-out estimation

The stock-out logic calculates average daily outgoing quantity from the preceding 30-day period, estimates days of stock remaining and reports items with 14 days of cover or less.

Purchase-order tracking

Purchase_Orders and Order_Items store order status, expected delivery, ordered quantity and received quantity. Pending-order logic calculates units still due and overdue days.

Warehouse comparison

Queries aggregate stock quantities and stock value to support comparisons across storage locations and product categories.

Expiry monitoring

Inventory_Batches stores expiry dates. The expiry view/query identifies batches approaching or passing their expiry dates.

Logistics analysis

Logistics records distance, transport method and cost. Queries compare transportation costs and delivery information.

Backup supplier lookup

Supplier-product relationships support finding alternative suppliers for products when the preferred supplier is unsuitable or inactive.

8. SQL Concepts Implemented

Concept	Implementation
DDL	CREATE DATABASE, CREATE TABLE, CREATE FUNCTION, CREATE VIEW and CREATE INDEX.
DML	INSERT, UPDATE and DELETE statements for adding and modifying records.
Keys and constraints	Primary keys, foreign keys, UNIQUE, NOT NULL, DEFAULT and CHECK constraints; ENUM columns for controlled status/type values.

Concept	Implementation
Joins	INNER JOIN, LEFT JOIN, RIGHT JOIN and CROSS JOIN examples.
Aggregation	COUNT, SUM, AVG, GROUP BY, HAVING and ORDER BY.
Subqueries	Nested and correlated subqueries, plus EXISTS-based filtering.
Window functions	DENSE_RANK and other ranking/analytic patterns in the SQL showcase.
Views	Saved SELECT statements for low stock, supplier ranking, order tracking, expiry alerts and stock-out forecast.
Indexing and optimisation	Indexes on movement type/date, purchase-order status/date, expiry date and warehouse-product movement history; EXPLAIN examples.
Transactions	START TRANSACTION, COMMIT, ROLLBACK and SAVEPOINT to demonstrate controlled changes.

9. Business Queries and Views

The SQL implementation includes five saved views:

- **v_low_stock:** lists stock below reorder level and calculates the shortfall.
- **v_supplier_ranking:** ranks supplier-product options using the implementation's supplier scoring logic.
- **v_order_tracking:** combines purchase-order and shipment information for tracking.
- **v_expiry_alerts:** highlights batches requiring expiry attention.
- **v_stock_out_forecast:** estimates stock coverage and potential stock-out timing.

The eight core business-query areas are:

- Low-stock detection and replenishment prioritisation.
- Best supplier identification and overall supplier leaderboard.
- Stock-out prediction for items with limited days of cover.
- Pending purchase orders, remaining units and overdue days.
- Supplier performance measured using on-time delivery information.
- Warehouse comparison and stock-value analysis.
- Replenishment tracking for quantities ordered but not yet received.
- Transportation analysis using shipment distance and transport cost.

Additional feature queries cover expiry monitoring, backup-supplier identification and order/shipment tracking. The query logic is intended to support decisions; it does not automatically send alerts or place orders.

10. Data Integrity and Verification

The script includes verification queries to check whether the sample database is internally consistent. These checks compare table row counts, reconcile inventory quantities against the stock-movement ledger, compare perishable batch totals with inventory quantities and identify warehouses whose stored unit totals exceed their defined capacity.

The ledger rule documented in the SQL file is: current inventory quantity equals total IN movements minus total OUT movements for the same warehouse and product. The verification queries are designed to return no discrepancy rows when the sample data is reconciled.

Constraints further protect the data. Examples include positive product prices, positive warehouse capacity, valid supplier quality scores, non-negative inventory quantities, received quantities not exceeding ordered quantities, and delivery dates not preceding dispatch dates.

11. Execution Guide

The project is provided as a single SQL script named **wholesale_distribution_system.sql**. Use MySQL 8.0+.

Option A — MySQL Workbench

- Open MySQL Workbench and connect to the MySQL server.
- Open wholesale_distribution_system.sql using File > Open SQL Script.
- Execute the script using the lightning-bolt execution control.
- Review the output/result tabs for table checks, views and query results.

Option B — Command line

```
mysql -u root -p < wholesale_distribution_system.sql
```

Important: The script contains `DROP DATABASE IF EXISTS wholesale_db` and then recreates the database. Running it again will delete and rebuild the existing wholesale_db database. Back up any important data before execution.

The function `fn_today()` returns the fixed date 2026-11-10 so that date-dependent sample outputs remain reproducible. For live-date behaviour, the SQL header notes that its body can be changed to return `CURDATE()`.

12. Limitations and Future Enhancements

- Develop a front-end interface for warehouse staff, procurement teams and administrators.
- Add authentication, role-based permissions and audit logging.
- Implement automatic low-stock notifications and purchase-order approval workflows.
- Replace the fixed reporting date with configurable system time for production use.
- Integrate supplier APIs and live shipment tracking where available.
- Extend stock-out estimation with seasonal demand, promotions and more historical data; validate forecast accuracy against actual outcomes.
- Add route optimisation using real road networks, delivery windows and vehicle capacity.
- Add automated scheduled backups, recovery testing and production monitoring.
- Introduce stored procedures or triggers for carefully controlled inventory updates, with tests for concurrency and partial deliveries.

13. Conclusion

SupplySync demonstrates how a relational database can support the core information needs of a wholesale distribution operation. Its 13-table schema connects suppliers, manufacturers, products, warehouses, customers, purchasing, inventory, batches, logistics and stock movements. The included views and SQL queries provide operational insight into replenishment, supplier choice, stock-out risk, pending orders, expiry and transportation costs.

The project also provides practical demonstrations of important DBMS topics, including keys and constraints, joins, aggregation, subqueries, window functions, views, indexes, query-plan inspection and transaction control. It is a

foundation for a fuller application; real-world deployment would require a user interface, security, automated workflows, operational testing and integration with external systems.

14. References

- Project implementation file: wholesale_distribution_system.sql (supplied with this project).
- MySQL 8.0 Reference Manual: https://dev.mysql.com/doc/refman/8.0/en/

Appendix: Key implementation facts

Database name: wholesale_db
DBMS: MySQL 8.0+
Storage engine: InnoDB
Number of relational tables: 13
Number of saved views: 5
Core business-query groups: 8
Reporting-date helper: fn_today() returns 2026-11-10